

Project requirements for Part 2

UI

Required screens (can be more):

- **Location area** view (map);
- Signal strength **list**;
- Screens to **add** and **edit** signal strengths.

Implementation requirements:

- Android Studio with Kotlin code;
- Single-activity architecture;
- Material design compliant UI;
- Navigation component;
- *RecyclerViews* or *Compose Lazy lists*.

Local data

Local **Room database** must be used to store:

- user-specific data (signal strengths);
- cache of signal strength map that is available from remote service.

Required architectural components:

- **LiveData** or Kotlin *Flow* for Room queries to be executed outside of the main thread (i.e. there is no UI lag during time consuming tasks like reading/writing to persistent storage);
- **ViewModel** – for UI data to survive configuration change (or share data between two fragments if you need it).

UI state should be preserved when app activities are killed by the system (can be tested with *Don't keep activities* enabled in device *Developer options*).

Remote data

Use supplied HTTP API **service** that provides signal strength map which is stored in the database of project **Part 1**. Room database must act as your app's local cache for this data.

Add **background task** for loading (i.e. populating cache) the signal strength map on the first launch of Android app. App should act appropriately if the map data is not (yet) available.

Retrofit (<https://square.github.io/retrofit/>) is recommended for calling HTTP API.

HTTP API endpoints:

- <http://localhost:9000/size> – map size;
- <http://localhost:9000/column?x=number> – map column with separate access points;
- <http://localhost:9000/wilibox-column?x=2> – map column with aggregated access points.